HOTEL MANAGEMENT SYSTEM

Deployment on Microsoft Azure

TEAM NUMBER : 03

CB.SC.U4AIE24116 - J. Lakshmi Tejaswi

CB.SC.U4AIE24130 - Vishnu Teja

CB.SC.U4AIE24135 - Mithul Akshay

CB.SC.U4AIE24136 - Mohan Innani

Introduction

What the system is, and why it moves to the cloud

- The Hotel Management System handles guest registration, room inventory, bookings, check-in and check-out, billing and payments.
- It is already built and working as a three-tier application: a Streamlit web front end, a Python data access layer, and a MySQL database of six normalised tables.
- Today it runs on a single machine. That machine is a single point of failure, is reachable from nowhere else, and has a capacity fixed by the hardware it sits on.
- This project migrates the system to Microsoft Azure so it becomes highly available, reachable from anywhere, and able to grow and shrink with demand.
- The whole environment is defined as code, so it can be created, destroyed and recreated identically rather than assembled by hand through a portal.

Scope of this project

The application logic is not rewritten. What changes is where it runs, who is responsible for the layers beneath it, and how it is provisioned. That is a deliberate choice: a migration is cheapest when the application was layered properly to begin with, which is what the database project established.

Problem Statement & Objectives

Six limitations of the local deployment, and what the cloud design sets out to fix

Single point of failure

Application and database share one machine. If it fails, the hotel cannot take a booking until it is repaired.

No remote access

Staff can only work at that one computer. A manager cannot check occupancy from another branch.

Fixed capacity

Hardware is sized for peak season and idles the rest of the year. A surge means buying a bigger machine.

Manual backups

Backups depend on someone remembering. A disk failure loses every guest and payment record.

Capital cost upfront

Servers and UPS must be bought before the first guest is served, and replaced every few years.

Maintenance burden

Patching the operating system, database and network is the hotel's problem, not a specialist's.

Objectives

Migrate to a public cloud · replace the self-hosted database with a managed service · scale elastically with demand · survive the loss of a datacentre · hold no credential in source code · define the whole environment as reproducible code

Existing System vs Proposed System

What actually changes when the application moves to Azure

ON-PREMISE (existing)	AZURE (proposed)
Application and database on one laptop	VM Scale Set plus managed MySQL service
Reachable only on that machine	Reachable from any browser, anywhere
Capacity fixed by the hardware bought	Autoscale adds instances on demand
Backups taken by hand, if at all	Automated daily backups, 7-day retention
Failure means downtime until repaired	Zone-redundant standby takes over
Large upfront capital expenditure	Pay-as-you-go operational expenditure
Database password sits in db_config.py	Password held in Key Vault, fetched at boot
Environment assembled manually, once	Whole environment defined in Bicep

The last two rows are the ones worth noticing: they are improvements the migration forces, not just relocations of the same thing.

Cloud Service Models

IaaS, PaaS and SaaS - and where each part of this project sits

IaaS

Infrastructure as a Service

The provider supplies virtual machines, storage and networking. We manage the operating system and everything above it.

In this project

Azure Virtual Machine Scale Sets run the application. We choose Ubuntu, install Python and Streamlit, and maintain them.

PaaS

Platform as a Service

The provider also manages the runtime or database engine. We supply only schema, configuration and data.

In this project

Azure Database for MySQL Flexible Server. Azure patches the engine, takes backups and handles failover; we design the schema.

SaaS

Software as a Service

The provider runs a finished application. The customer opens a browser and uses it, owning nothing.

In this project

The finished system is itself delivered this way to hotel staff, who visit a URL and own no infrastructure at all.

The project deliberately uses all three levels. The higher up the stack a component sits, the less we manage and the less we control.

Cloud Deployment Models

Why a public cloud, and when it would be the wrong answer

Public cloud

CHOSEN

Infrastructure owned by a provider and shared among many tenants. Lowest cost, nothing to buy, effectively unlimited capacity.

Private cloud

Infrastructure dedicated to one organisation. More control and easier regulatory compliance, but the capital cost returns.

Hybrid cloud

A combination: sensitive data kept private, general workloads in the public cloud. Flexible, but harder to operate.

Community cloud

Shared by organisations with common requirements, such as a hotel group or a university consortium.

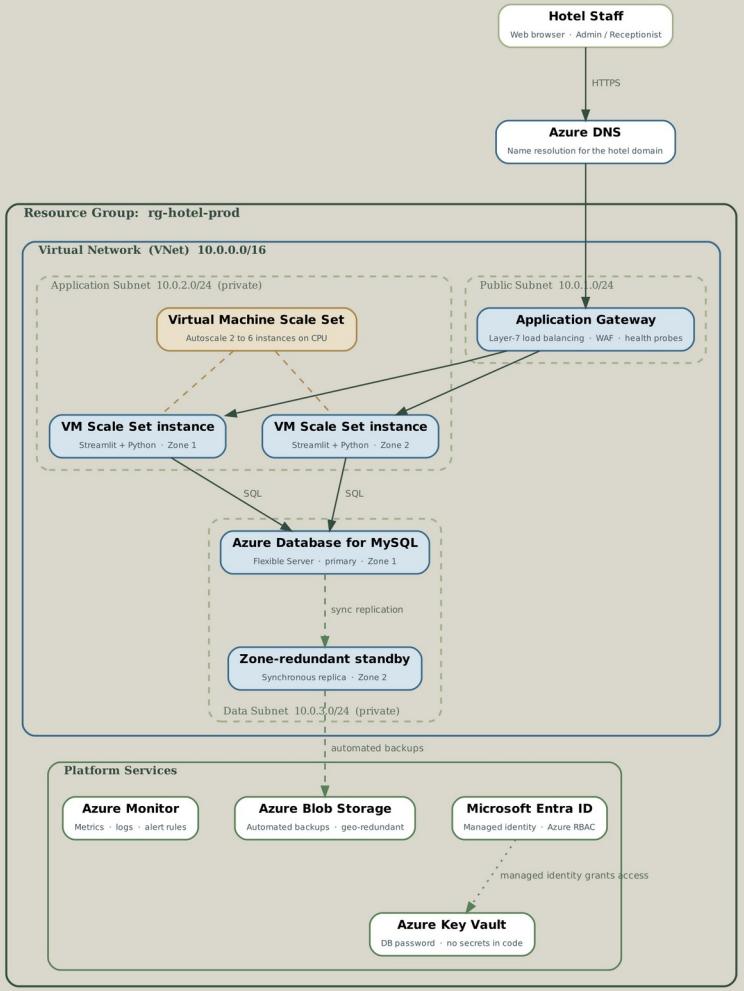
Why public cloud

- No capital expenditure; runs on an Azure for Students account
- Availability Zones give datacentre redundancy with no extra work
- Resources can be created and destroyed as the project needs them
- A single hotel has no regulation forcing data to stay on its own premises
- The skills transfer directly to industry

When it would be wrong: a hotel chain legally required to keep guest identity documents inside its own country, or on its own hardware, would need a private or hybrid arrangement instead.

Proposed Cloud Architecture

Three tiers inside one virtual network, spread across two Availability Zones



Entry point

Azure DNS resolves the hotel's domain to an Application Gateway, which load-balances across healthy instances and stops routing to any that fail a probe.

Application tier

A Virtual Machine Scale Set in a private subnet runs the Streamlit application, spread across Zone 1 and Zone 2, autoscaling between two and six instances.

Data tier

Azure Database for MySQL Flexible Server holds the six tables, VNet-integrated so it has no public endpoint at all. A zone-redundant standby can take over automatically.

Secrets

The database password lives in Azure Key Vault. Each VM reads it at boot using its managed identity, so no credential exists in the repository or the VM image.

Operations

Azure Monitor collects metrics, logs and alert rules; Blob Storage holds automated database backups.

Azure Services Used

Each service, and the cloud concept it demonstrates

Virtual Machine Scale Sets
Run the Streamlit application
IaaS, elasticity

Application Gateway v2
Layer-7 load balancing and health probes
Load balancing

Network Security Groups
Per-tier firewall rules
Defence in depth

Microsoft Entra ID
Managed identity and Azure RBAC
Identity and access

Azure Blob Storage
Automated database backups
Durable object storage

Azure Bicep
Declares the whole environment as code
Infrastructure as code

Azure Database for MySQL
Managed database, Flexible Server
PaaS, managed services

Azure Virtual Network
Private network with three subnets
Network isolation

Azure Key Vault
Stores the database password
Secrets management

Azure Monitor
Metrics, logs and alert rules
Observability

Availability Zones
Two physically separate datacentres
Fault tolerance

Azure for Students
Free credit, cost control
Pay-as-you-go pricing

Infrastructure as Code

The entire environment is declared in Azure Bicep, not clicked together in the portal

```
// modules/compute.bicep  -  autoscale rule
resource autoscale 'Microsoft.Insights/autoscalesettings@2022-10-01' = {
  properties: {
    profiles: [ {
      capacity: { minimum: '2', maximum: '6', default: '2' }
      rules: [ {
        metricTrigger: {
          metricName: 'Percentage CPU'
          timeWindow:  'PT5M'
          operator:    'GreaterThan'
          threshold:   70
        }
        scaleAction: {
          direction: 'Increase'
          value:     '1'
          cooldown:  'PT5M'
        } } ] } ] } }
```

Why this matters

The environment can be destroyed after a demonstration and rebuilt identically in one command. Nothing depends on remembering which buttons were pressed, and the architecture is reviewable in version control like any other code.

main.bicep	Orchestrates all six modules and wires their outputs together
modules/network.bicep	VNet, three subnets, three network security groups
modules/database.bicep	MySQL Flexible Server, private DNS zone, HA options
modules/keyvault.bicep	Key Vault and the database password secret
modules/gateway.bicep	Application Gateway, public IP, health probe
modules/compute.bicep	Scale set, managed identity, autoscale rules
modules/monitoring.bicep	Log Analytics workspace and alert rules
cloud-init.yaml	Bootstraps each VM: packages, secret, systemd service
deploy.sh / destroy.sh	One command to build the stack, one to remove it

All seven Bicep files compile to a valid ARM template with no errors or warnings; the shell scripts pass shellcheck and the cloud-init document parses as valid YAML.

Database Migration Strategy

Local MySQL to Azure Database for MySQL, without touching a single query

1. **Export** — Dump schema and data with mysqldump: six tables, three views, two triggers and one stored procedure in a single file.
2. **Provision** — Create the Flexible Server inside the delegated data subnet, with a private DNS zone so it has no public endpoint.
3. **Configure** — Set log_bin_trust_function_creators to ON, so the triggers and stored procedure can be created without SUPER privilege.
4. **Import** — Load the dump from inside the VNet. The schema transfers unchanged because the service runs genuine MySQL 8.0.
5. **Repoint** — db_config.py is written at boot from Key Vault, pointing at the Azure hostname. No application code changes.
6. **Verify** — Re-run the CRUD operations and the four reports, and compare row counts against the source database.

Because every SQL statement lives in database.py and every credential in db_config.py, moving the database is a configuration change rather than a code change.

Step 3 is the one that catches people

Azure enables binary logging by default. With it on, creating a trigger or stored procedure needs a privilege the administrator account does not have, and the import fails partway through.

Setting that one server parameter first is the difference between a clean import and a confusing error.

It is declared in database.bicep, so it is applied before anyone tries the import.

Scalability and Elasticity

How the deployment copes with a busy season

Vertical scaling (scale up)

Give the existing machine more CPU or memory. Simple, but there is a ceiling and the machine usually restarts.

Used for the database: the server can be resized to a larger tier if queries slow down.

Horizontal scaling (scale out)

Add more machines and share the load between them. No practical ceiling, and instances join while the system runs.

Used for the application tier: the scale set adds instances automatically.

The autoscale rules, as deployed

Normal load	Two instances, one in each Availability Zone
Average CPU above 70% for 5 minutes	One instance added, up to a maximum of six
Average CPU below 30% for 10 minutes	One instance removed, never below two
Instance fails its health probe	Gateway stops routing to it; the scale set replaces it
Cooldown	5 minutes after scaling out, 10 after scaling in, so one burst does not cause repeated scaling

Scalability is the ability to grow; elasticity is doing it automatically, in both directions, in response to live demand.

High Availability and Fault Tolerance

What happens when something breaks

One application instance fails

The gateway's health probe stops routing to it within seconds, and the scale set launches a replacement. Guests notice nothing.

An entire Availability Zone fails

The instance and the database standby in the other zone keep serving. Capacity halves; the system stays up.

The primary database fails

Azure promotes the zone-redundant standby automatically. The server hostname does not change, so the application reconnects without a code change.

Data is deleted by mistake

Restore from an automated backup. Point-in-time restore covers the retention window, which is seven days by default.

Recovery objectives

RTO - recovery time objective - is how long the hotel can be down; zone failover completes in minutes.

RPO - recovery point objective - is how much data may be lost; synchronous replication makes this effectively zero for a zone failure.

An honest limitation

Zone-redundant database HA requires a General Purpose tier. The Burstable tier a student subscription can afford does not support it, so our deployed configuration runs without database HA while the design provides for it. The template takes a flag and reports which mode is actually active.

Cloud Security

Layered controls, from the network inward

Network isolation

The VNet gives a private network. Only the Application Gateway sits in a public subnet; the VMs have no public IP and the database no public endpoint.

Network security groups

One per tier. The database accepts port 3306 only from the application subnet; the application accepts 8501 only from the gateway subnet.

No secrets in code

The database password lives in Key Vault. Each VM reads it at boot with its managed identity, so nothing is committed to the repository.

Managed identity and RBAC

The scale set's identity is granted exactly one role - Key Vault Secrets User - and nothing else. No keys exist to leak.

Encryption

TLS in transit, enforced on MySQL connections; Azure encrypts the database and backups at rest. Guest ID and contact details are covered by both.

Shared responsibility

Azure secures the cloud itself - hardware, hypervisor, managed services. We secure what we put in it - data, access rules, configuration.

Carried over from the database project: passwords are stored as SHA-256 hashes and every query is parameterised, so the application is not vulnerable to SQL injection.

Monitoring and Cost Model

Watching the system, and paying only for what runs

Monitored by Azure Monitor

CPU utilisation	Drives the autoscale rules and a high-CPU alert
Health probe status	Shows instances the gateway has taken out of rotation
Database connections	Warns before the connection limit is reached
Storage used	Alerts before the server runs out of space
Application logs	Collected centrally, so failures are traceable without SSH

Pay-as-you-go cost model

Compute	Billed per second a VM runs, so scaling in at night costs nothing to do
Database	Billed per hour by tier, plus the storage consumed
Gateway	Billed hourly plus capacity units; the largest fixed cost in this design
Storage	Per GB per month; backup snapshots are incremental
Egress	Inbound free, outbound charged per GB

Capital expenditure versus operational expenditure

Buying a server is a capital expense: paid once, owned, and depreciating whether used or not. Cloud is operational: paid monthly, matched to usage, and stopped the moment a resource is switched off. For a hotel with a seasonal occupancy pattern the second model fits demand far more closely.

It is not automatically cheaper. For a steady, always-on workload with predictable load, owning hardware can work out less expensive over several years. The cloud wins on variable demand, on not paying before you start, and on not having to replace anything.

Work Completed and Implementation Plan

What exists today, and what happens next

Completed

- ✓ Three-tier application built and working: Streamlit front end, Python data access layer, MySQL database
- ✓ Six normalised tables in 3NF with foreign keys, three views, two triggers and a stored procedure
- ✓ Cloud architecture designed: network layout, service selection, scaling policy and security model
- ✓ Infrastructure as Code written: seven Bicep files, cloud-init bootstrap, deploy and teardown scripts
- ✓ Templates validated - the full stack compiles to a valid ARM template with no errors or warnings
- ✓ Migration procedure documented, including the binary-logging parameter the import depends on

Remaining

Phase 1	Provision the resource group and run the deployment
Phase 2	Load the schema and verify row counts against the local database
Phase 3	Confirm the gateway reports both instances healthy
Phase 4	Load-test the autoscale rules and capture the instance count rising
Phase 5	Record the running cost, then tear the environment down

Stated plainly

The environment has not been provisioned yet. What exists is a complete, validated infrastructure definition that creates it in a single command. We chose to invest the time in making the deployment reproducible rather than clicking through the portal once and screenshotting the result.

Challenges and Considerations

Problems we had to design around, stated honestly

Streamlit holds session state in memory

A user sent to a different instance mid-session would be logged out. Solved with cookie-based session affinity on the Application Gateway.

Burstable tiers cannot do HA

The affordable database tier does not support zone redundancy. The template takes a flag and reports which mode is actually active.

The database has no public endpoint

Deliberate, but it means the schema cannot be loaded from a laptop. We load it by running a command on an instance already inside the VNet.

Cost can run away quietly

An idle Application Gateway bills by the hour. Mitigated with a teardown script, a capped instance count and a budget alert.

Binary logging blocks trigger creation

Azure MySQL needs log_bin_trust_function_creators set before the import, or it fails partway through.

Vendor lock-in

Managed services make moving provider harder. Mitigated by keeping standard MySQL and plain Python, so only the infrastructure layer is Azure-specific.

Every item here was found while writing the templates, not during the presentation. Each one has a mitigation already in the code.

Conclusion and Future Scope

Always available

Spreading across two zones removes the single point of failure the laptop version had.

Reachable anywhere

Staff open a browser; nothing is installed on any hotel computer.

Follows demand

Capacity rises for the season and falls again, with no one watching.

Credentials protected

Key Vault and managed identity remove the password from source control.

Reproducible

The environment is code, so it can be destroyed and rebuilt identically.

Predictable spend

Monthly operating cost in place of an upfront hardware purchase.

Future scope

- Containerise the application and run it on Azure Kubernetes Service
- Add read replicas so reporting queries do not compete with bookings
- Introduce a CI/CD pipeline in GitHub Actions, so a commit deploys itself
- Extend to a second region for disaster recovery beyond a single geography

Conclusion

A working three-tier database application has been designed for migration to Microsoft Azure, using IaaS for compute and PaaS for data, spread across two Availability Zones, and defined entirely as infrastructure code that compiles and validates.

The application itself is unchanged - which is the clearest evidence that separating the data access layer from the interface in the database project was the right decision.

THANK YOU

Questions welcome